



Gobierno Bolivariano
de Venezuela

Ministerio del Poder Popular
para la Educación Universitaria

Universidad Nacional Experimental
para las Telecomunicaciones e Informática (UNETI)



República Bolivariana de Venezuela.

Ministerio del Poder Popular para la Educación Universitaria.

Universidad Nacional Experimental de las Telecomunicaciones e Informática.

(UNETI).

Trayecto III.

Sección: 6y7A

Unidad Curricular: Programación III (M2) (Informática)

Evaluación 2

Alumno:

Ender Muñoz

C.I: 30.296.071

Profesor:

Carlos Márquez



Introducción

En la primera evaluación construimos una aplicación base con Ionic y Angular que tenía tres páginas: Inicio (con temática del Clásico Mundial de Béisbol 2026), Información Personal (mis datos y perfil profesional) y Contacto (una vista simple con un botón a GitHub).

Para esta segunda evaluación, el reto fue transformar esa página de contacto en algo mucho más funcional. La idea era poner en práctica la creación de **Formularios**, la persistencia de datos y el manejo de estado en el cliente.

El objetivo era pasar de una página estática a una herramienta que permitiera al usuario registrar comentarios, verlos en una lista, filtrarlos con búsqueda, seleccionar varios elementos y eliminarlos de forma selectiva, todo sincronizado con el **localStorage** del navegador.

Con estas ideas, le agregué a la página de contacto:

- Un formulario reactivo con validaciones en tiempo real.
- Almacenamiento local para guardar los comentarios.
- Una lista dinámica con selección individual y masiva.
- Una barra de búsqueda que filtra al instante.
- Un botón de eliminar con confirmación que solo borra los marcados.
- Notificaciones Toast para informar sobre cada acción.



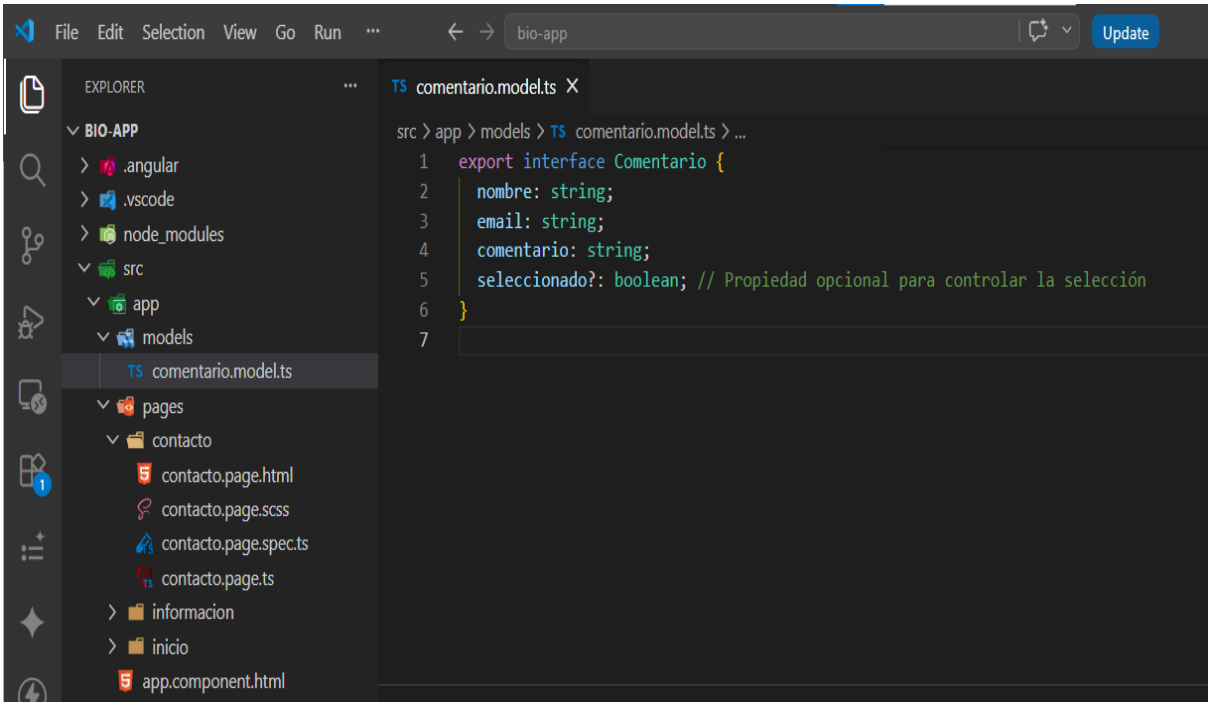
Desarrollo

En esta sección voy a contar el paso a paso de todo lo que agregué y mejoré en la página de contacto. Cada uno de estos cambios tiene su lógica y su propósito, y lo fui documentando con capturas de pantalla para que se vea el proceso.

1. Creación del modelo de datos (Comentario)

Este archivo se usó para poder mapear y controlar el check, y así poder seleccionar uno o más datos a eliminar. Fue clave porque me permitió manejar el estado de cada comentario de forma individual, añadiendo esa propiedad **seleccionado** que me sirvió después para saber cuáles registros estaban marcados y cuáles no.

comentario.model.ts



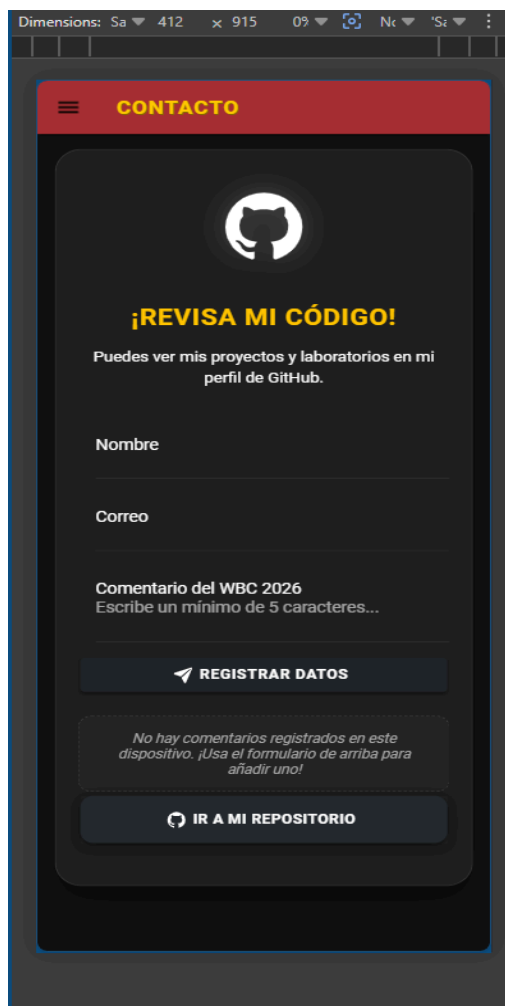
```
src > app > models > TS comentario.model.ts > ...
1  export interface Comentario {
2      nombre: string;
3      email: string;
4      comentario: string;
5      seleccionado?: boolean; // Propiedad opcional para controlar la selección
6  }
7
```

2. Configuración del formulario

En el componente **ContactoPage** se inyectó **FormBuilder** y se creó el **FormGroup** con sus respectivos validadores:

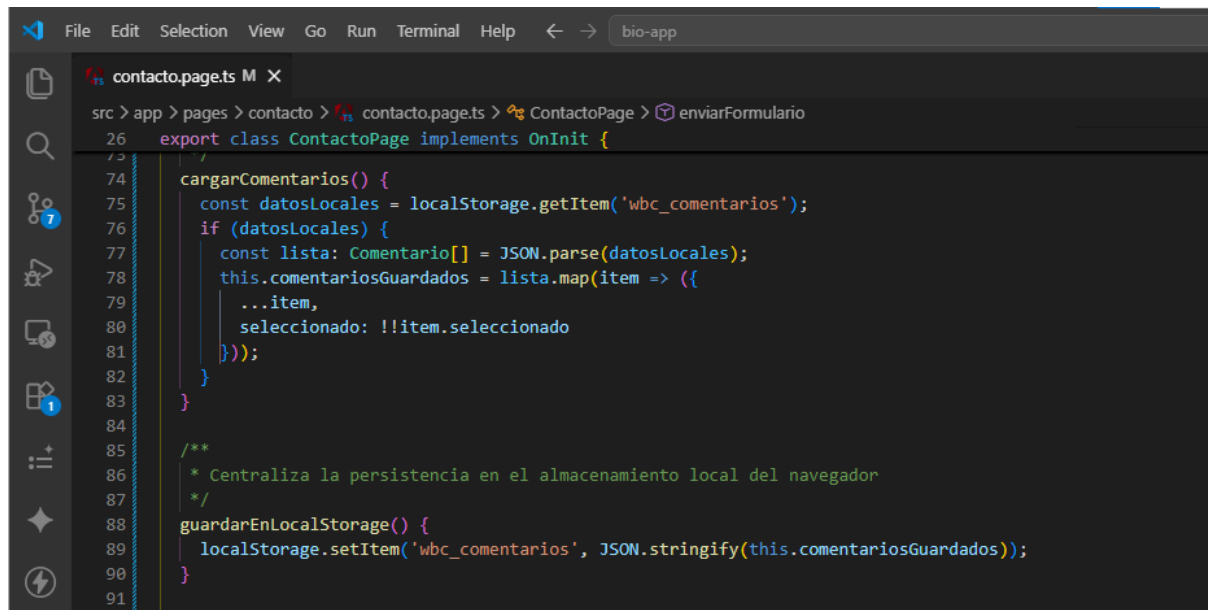
```
File Edit Selection View Go Run Terminal Help bio-app
contacto.page.ts M X
src > app > pages > contacto > contacto.page.ts > ContactoPage > alertButtons
26 export class ContactoPage implements OnInit {
53   constructor() {
54     // Inicializo los íconos de GitHub, el avión de papel y la basura
55     addIcons({ logoGithub, paperPlane, trash });
56   }
57
58   ngOnInit() {
59     // Definimos las validaciones estrictas del formulario reactivo
60     this.wbcForm = this.formBuilder.group({
61       nombre: ['', [Validators.required, Validators.minLength(3)]],
62       email: ['', [Validators.required, Validators.email]],
63       comentario: ['', [Validators.required, Validators.minLength(5)]]
64     });
65
66     // Cargamos comentarios previos si existen en el LocalStorage
67     this.cargarComentarios();
68   }
69 }
```

Imagen de la interfaz, donde se puede visualizar el formulario creado



3. Persistencia en LocalStorage

Se implementaron dos métodos para gestionar el almacenamiento local:

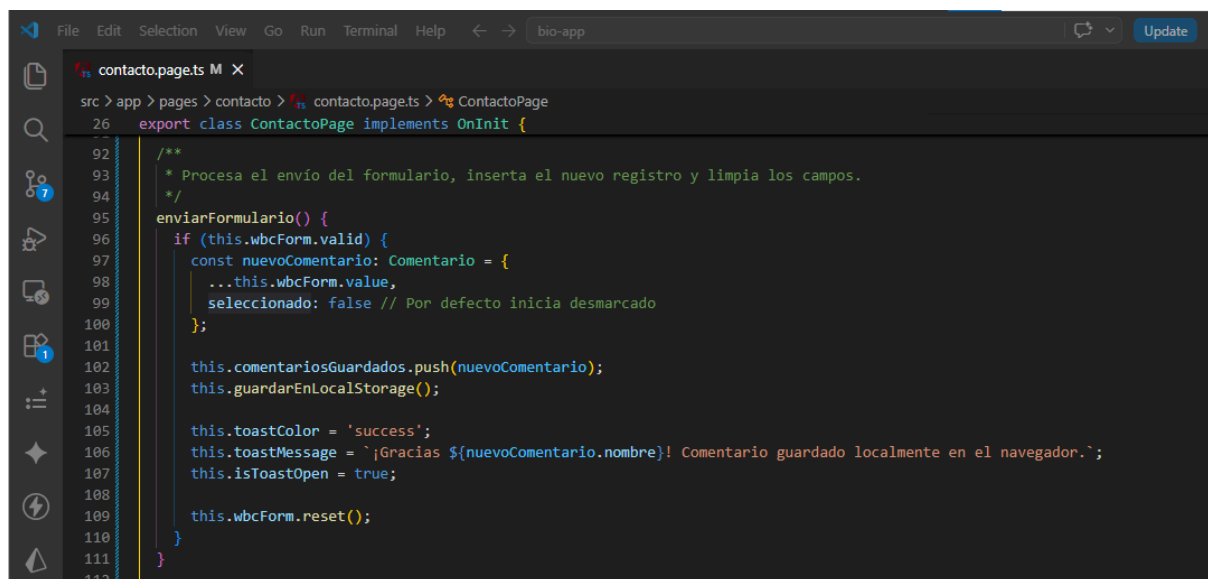


```
File Edit Selection View Go Run Terminal Help < -> bio-app
contacto.page.ts M X
src > app > pages > contacto > contacto.page.ts > ContactoPage > enviarFormulario
26 export class ContactoPage implements OnInit {
73
74 cargarComentarios() {
75   const datosLocales = localStorage.getItem('wbc_comentarios');
76   if (datosLocales) {
77     const lista: Comentario[] = JSON.parse(datosLocales);
78     this.comentariosGuardados = lista.map(item => ({
79       ...item,
80       seleccionado: !!item.seleccionado
81     }));
82   }
83 }
84
85 /**
86  * Centraliza la persistencia en el almacenamiento local del navegador
87  */
88 guardarEnLocalStorage() {
89   localStorage.setItem('wbc_comentarios', JSON.stringify(this.comentariosGuardados));
90 }
91
```

Al iniciar la página, se invoca **cargarComentarios()** para recuperar los datos previos. Cada vez que se añade o elimina un comentario, se actualiza el almacenamiento con **guardarEnLocalStorage()**.

4. Envío del formulario y creación de comentarios

Este método se encarga de validar que todo esté correcto, crear un nuevo comentario con la propiedad **seleccionado** en **false** (para que empiece desmarcado), guardarlo en el arreglo y también en el **localStorage**, y finalmente mostrar un mensaje de éxito y limpiar el formulario.



```
File Edit Selection View Go Run Terminal Help < -> bio-app Update
contacto.page.ts M X
src > app > pages > contacto > contacto.page.ts > ContactoPage
26 export class ContactoPage implements OnInit {
92
93 /**
94  * Procesa el envío del formulario, inserta el nuevo registro y limpia los campos.
95  */
96 enviarFormulario() {
97   if (this.wbcForm.valid) {
98     const nuevoComentario: Comentario = {
99       ...this.wbcForm.value,
100       seleccionado: false // Por defecto inicia desmarcado
101     };
102     this.comentariosGuardados.push(nuevoComentario);
103     this.guardarEnLocalStorage();
104
105     this.toastColor = 'success';
106     this.toastMessage = `¡Gracias ${nuevoComentario.nombre}! Comentario guardado localmente en el navegador.`;
107     this.isToastOpen = true;
108
109     this.wbcForm.reset();
110   }
111 }
112
```



Después de guardar, se muestra un mensaje de éxito y se limpia el formulario.

Dimensions: Sa 412 x 915 0% Nc 'Sz

CONTACTO



¡REVISA MI CÓDIGO!

Puedes ver mis proyectos y laboratorios en mi perfil de GitHub.

Nombre

Ender Muñoz

Correo

endermunoz18@gmail.com

Comentario del WBC 2026

Texto Prueba

REGISTRAR DATOS

No hay comentarios registrados en este dispositivo. ¡Usa el formulario de arriba para añadir uno!

IR A MI REPOSITORIO

Dimensions: Sa 412 x 915 0% Nc 'Sz

¡Gracias Ender Muñoz! Comentario guardado localmente en el navegador.



¡REVISA MI CÓDIGO!

Puedes ver mis proyectos y laboratorios en mi perfil de GitHub.

Nombre

Correo

Comentario del WBC 2026

Escribe un mínimo de 5 caracteres...

REGISTRAR DATOS

REGISTROS LOCALES (1)

☐ Seleccionar todos los registros

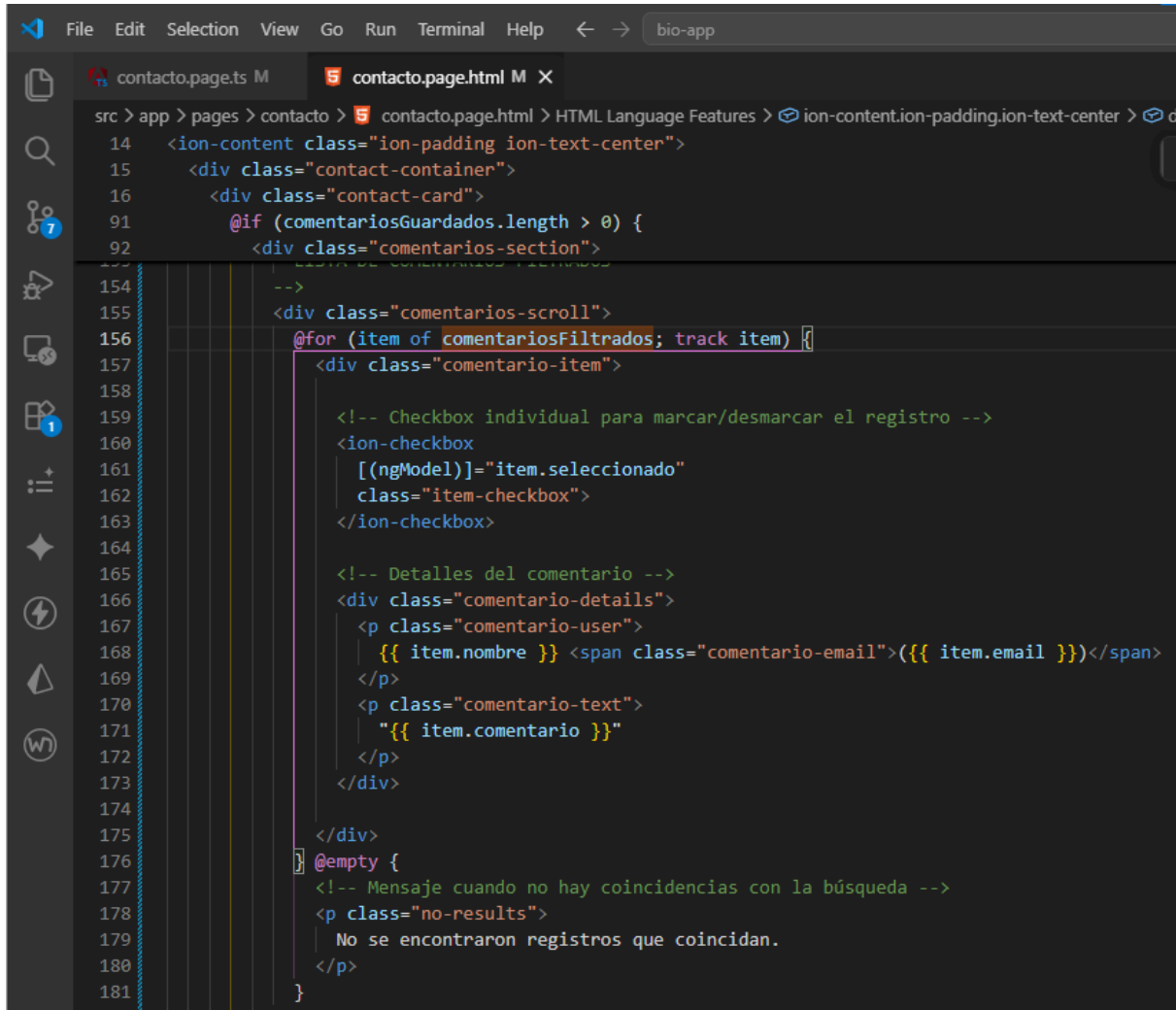
☒ Ender Muñoz (endermunoz18@gmail.com)

"Texto Prueba"

IR A MI REPOSITORIO

5. Lista de comentarios y selección múltiple

En el HTML recorrí el arreglo **comentariosFiltrados** y por cada registro mostré un checkbox con **[(ngModel)]** enlazado a la propiedad **seleccionado**. Así puedo marcar o desmarcar cada comentario individualmente.



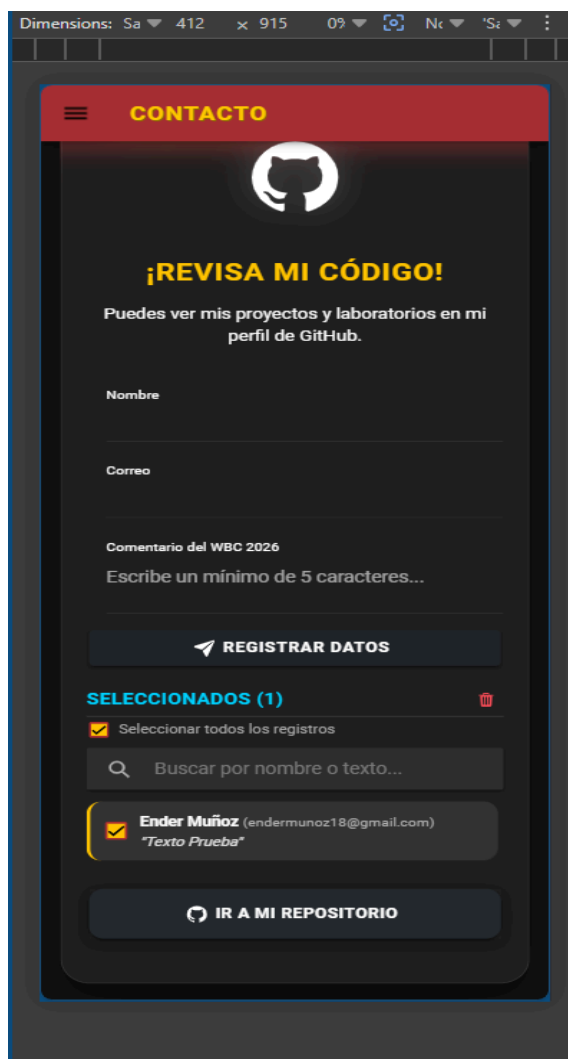
```
src > app > pages > contacto > contacto.page.html > HTML Language Features > ion-content.ion-padding.ion-text-center > c
14 <ion-content class="ion-padding ion-text-center">
15 <div class="contact-container">
16 <div class="contact-card">
91 <!-- Comentarios Guardados -->
92 <div class="comentarios-section">
154 <!--
155 <div class="comentarios-scroll">
156 <!-- @for (item of comentariosFiltrados; track item) -->
157 <div class="comentario-item">
158
159 <!-- Checkbox individual para marcar/desmarcar el registro -->
160 <ion-checkbox
161 <!-- [(ngModel)]="item.seleccionado" -->
162 class="item-checkbox">
163 </ion-checkbox>
164
165 <!-- Detalles del comentario -->
166 <div class="comentario-details">
167 <p class="comentario-user">
168 <!-- {{ item.nombre }} --> <span class="comentario-email">{{ item.email }}</span>
169 </p>
170 <p class="comentario-text">
171 <!-- {{ item.comentario }} -->
172 </p>
173 </div>
174
175 </div>
176 <!-- @empty -->
177 <!-- Mensaje cuando no hay coincidencias con la búsqueda -->
178 <p class="no-results">
179 <!-- No se encontraron registros que coincidan. -->
180 </p>
181 </div>
182 </div>
183 </ion-content>
```

También agregué un checkbox maestro que permite seleccionar o deseleccionar todos los registros a la vez. Este se enlaza con el getter **todosSeleccionados** para reflejar el estado general, y al cambiar ejecuta el método **seleccionarTodos** que actualiza todos los checkboxes.



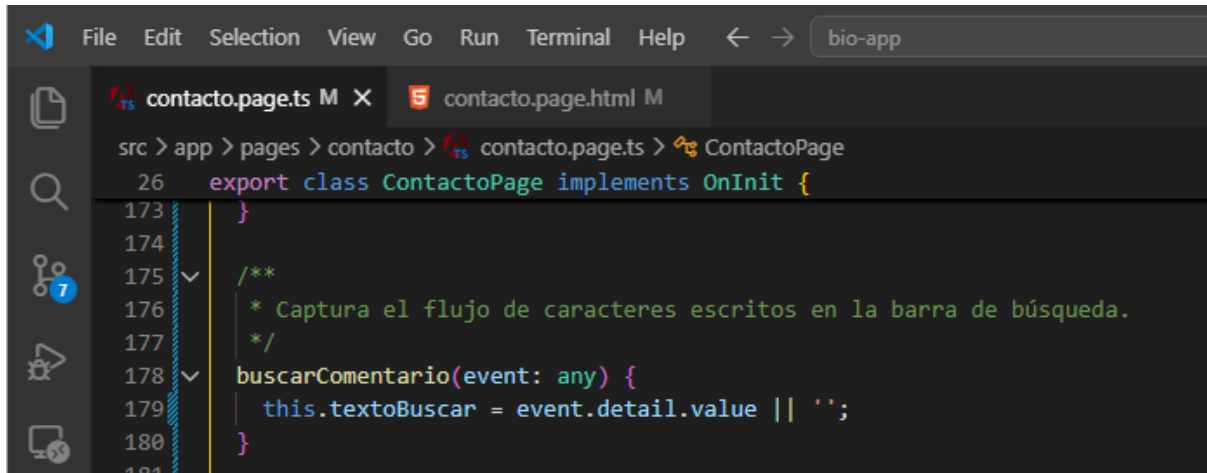
```
File Edit Selection View Go Run Terminal Help ← → bio-app
contacto.page.ts M contacto.page.html M X
> app > pages > contacto > contacto.page.html > HTML Language Features > ion-content.ion-padding.ion-text-center > div
14 <ion-content class="ion-padding ion-text-center">
15 <div class="contact-container">
16 <div class="contact-card">
91 @if (comentariosGuardados.length > 0) {
92 <div class="comentarios-section">
124 (didDismiss)="isAlertOpen = false">
125 </ion-alert>
126
127 <!--
128 CHECKBOX PARA SELECCIONAR TODOS
129 - Se enlaza con el getter 'todosSeleccionados' para reflejar el estado.
130 - Al cambiar, se ejecuta 'seleccionarTodos($event)' que marca/desmarca todos.
131 -->
132 <div class="select-all-container">|
133 <ion-checkbox
134 [checked]="todosSeleccionados"
135 (ionChange)="seleccionarTodos(event: $event)"
136 class="select-all-checkbox">
137 </ion-checkbox>
138 <span class="select-all-label">Seleccionar todos los registros</span>
139 </div>
140
```

Interfaz mostrando el checkboxes



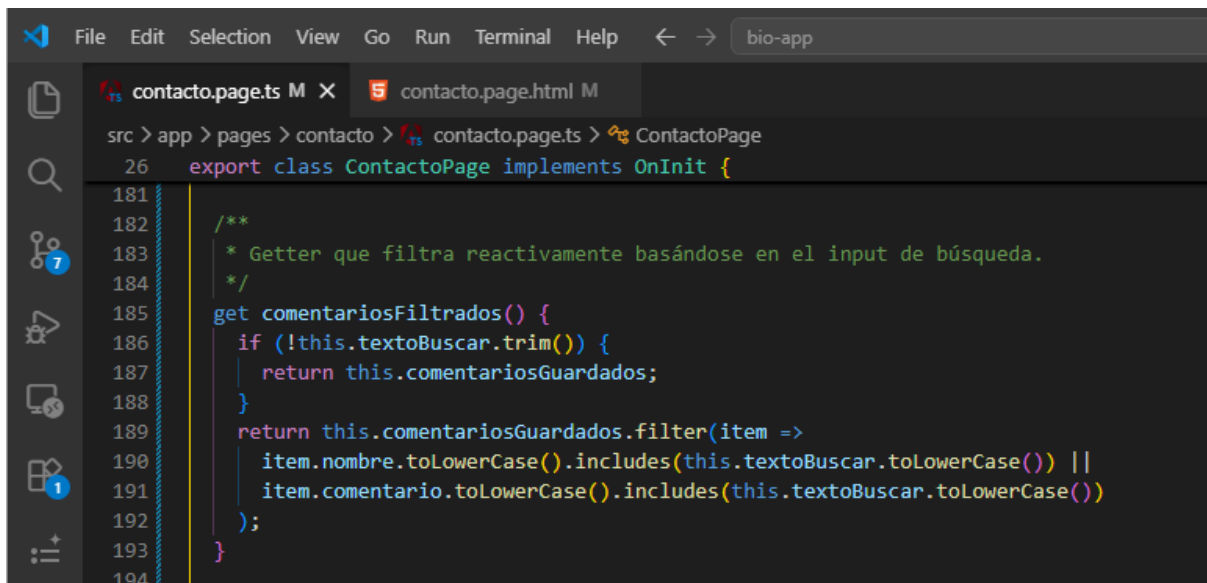
6. Búsqueda en tiempo real

La barra de búsqueda tiene un evento **ionInput** que ejecuta el método **buscarComentario**. Este método captura lo que el usuario escribe y lo guarda en la variable **textoBuscar**.

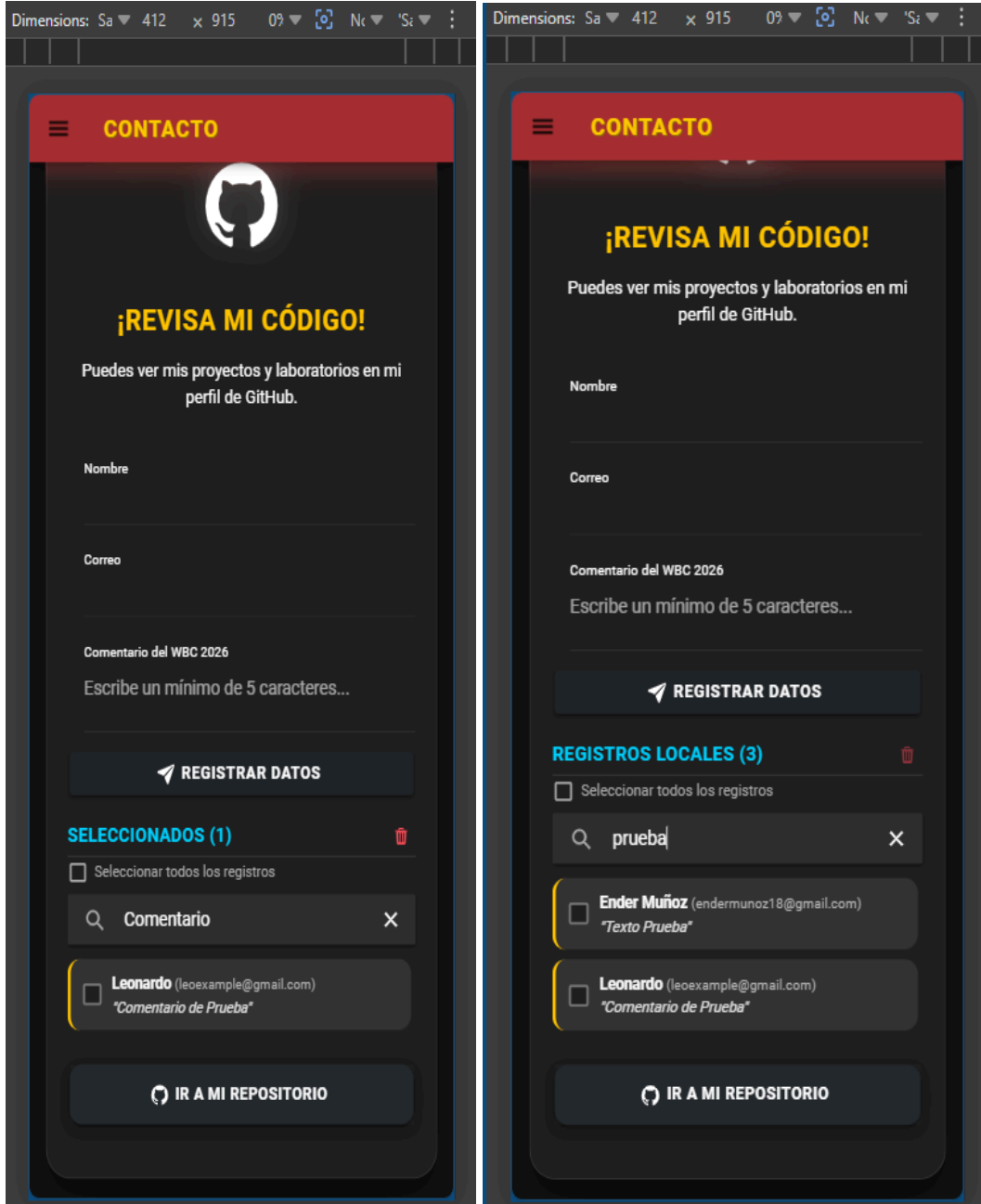


```
File Edit Selection View Go Run Terminal Help bio-app
contacto.page.ts M X contacto.page.html M
src > app > pages > contacto > contacto.page.ts > ContactoPage
26 export class ContactoPage implements OnInit {
173 }
174
175 /**
176  * Captura el flujo de caracteres escritos en la barra de búsqueda.
177  */
178 buscarComentario(event: any) {
179   this.textoBuscar = event.detail.value || '';
180 }
181
```

Luego creé un getter llamado **comentariosFiltrados** que revisa el texto ingresado y filtra los comentarios comparando tanto el nombre como el contenido del comentario. Si no hay texto, simplemente muestra todos los registros.

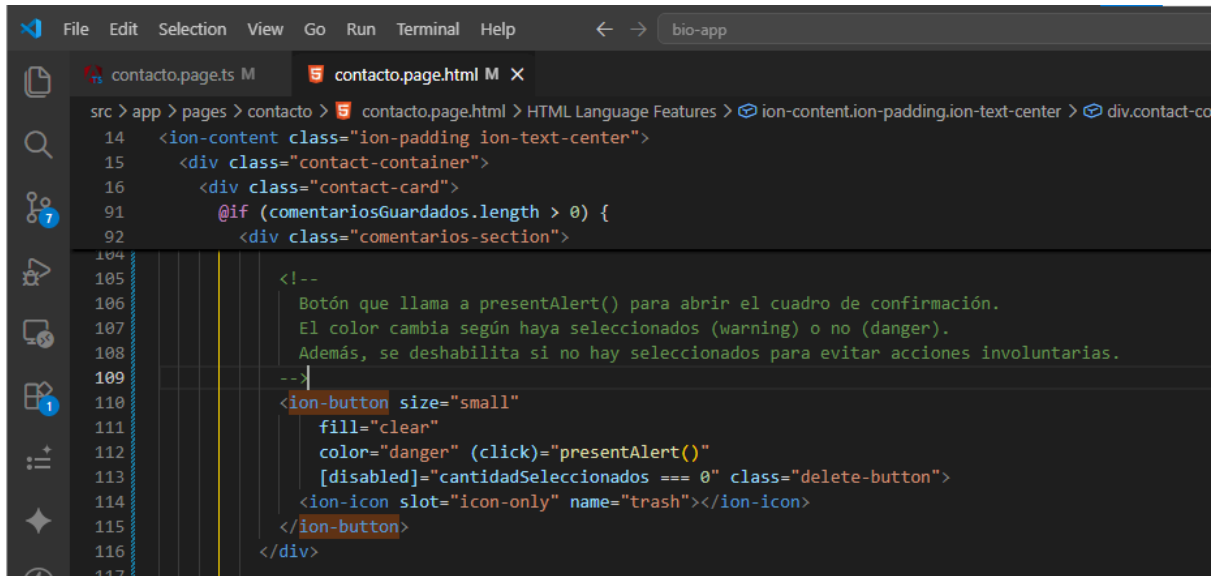


```
File Edit Selection View Go Run Terminal Help bio-app
contacto.page.ts M X contacto.page.html M
src > app > pages > contacto > contacto.page.ts > ContactoPage
26 export class ContactoPage implements OnInit {
181
182 /**
183  * Getter que filtra reactivamente basándose en el input de búsqueda.
184  */
185 get comentariosFiltrados() {
186   if (!this.textoBuscar.trim()) {
187     return this.comentariosGuardados;
188   }
189   return this.comentariosGuardados.filter(item =>
190     item.nombre.toLowerCase().includes(this.textoBuscar.toLowerCase()) ||
191     item.comentario.toLowerCase().includes(this.textoBuscar.toLowerCase())
192   );
193 }
194
```



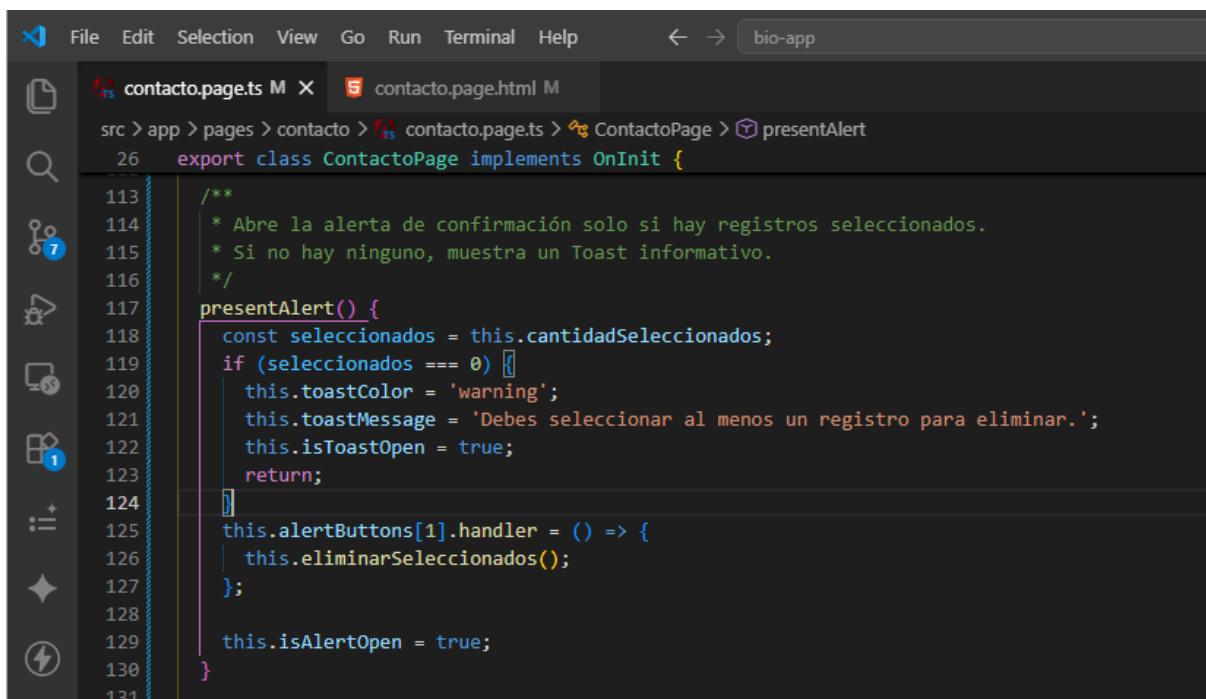
7. Eliminación selectiva con confirmación

El botón de papelera se deshabilita automáticamente cuando no hay registros seleccionados, así se evita que el usuario intente eliminar sin haber marcado nada.



```
src > app > pages > contacto > contacto.page.html > HTML Language Features > ion-content.ion-padding.ion-text-center > div.contact-co
14 <ion-content class="ion-padding ion-text-center">
15 <div class="contact-container">
16 <div class="contact-card">
91 @if (comentariosGuardados.length > 0) {
92 <div class="comentarios-section">
104
105 <!--
106 Botón que llama a presentAlert() para abrir el cuadro de confirmación.
107 El color cambia según haya seleccionados (warning) o no (danger).
108 Además, se deshabilita si no hay seleccionados para evitar acciones involuntarias.
109 -->
110 <ion-button size="small"
111 fill="clear"
112 color="danger" (click)="presentAlert()"
113 [disabled]="cantidadSeleccionados === 0" class="delete-button">
114 <ion-icon slot="icon-only" name="trash"></ion-icon>
115 </ion-button>
116 </div>
117
```

Al hacer clic, se ejecuta **presentAlert()**. Este método primero verifica si hay comentarios seleccionados. Si no hay, muestra un Toast de advertencia y no abre la alerta. Si hay, configura el mensaje y abre la ventana de confirmación.



```
src > app > pages > contacto > contacto.page.ts > ContactoPage > presentAlert
26 export class ContactoPage implements OnInit {
113
114 /**
115  * Abre la alerta de confirmación solo si hay registros seleccionados.
116  * Si no hay ninguno, muestra un Toast informativo.
117  */
118 presentAlert() {
119 const seleccionados = this.cantidadSeleccionados;
120 if (seleccionados === 0) {
121 this.toastColor = 'warning';
122 this.toastMessage = 'Debes seleccionar al menos un registro para eliminar.';
123 this.isToastOpen = true;
124 return;
125 }
126 this.alertButtons[1].handler = () => {
127 this.eliminarSeleccionados();
128 };
129 this.isAlertOpen = true;
130 }
131
```

Si el usuario confirma, se ejecuta **eliminarSeleccionados()**. Este método filtra el arreglo y se queda solo con los comentarios que NO están marcados, elimina los seleccionados y guarda



los cambios en el **localStorage**. Después muestra un Toast indicando cuántos registros fueron borrados.

```
File Edit Selection View Go Run Terminal Help bio-app
contacto.page.ts M X contacto.page.html M
src > app > pages > contacto > contacto.page.ts > ContactoPage > presentAlert
26 export class ContactoPage implements OnInit {
132
133 /**
134  * Remueve únicamente los comentarios individuales que tengan 'seleccionado' en true.
135  */
136 eliminarSeleccionados() {
137     const inicial = this.comentariosGuardados.length;
138     // Nos quedamos sólo con los que NO están marcados
139     this.comentariosGuardados = this.comentariosGuardados.filter(item => !item.seleccionado);
140     const totalAAborrar = inicial - this.comentariosGuardados.length;
141
142     this.guardarEnLocalStorage();
143     this.textoBuscar = '';
144
145     this.toastColor = 'danger';
146     this.toastMessage = totalAAborrar === 1
147       ? 'El registro seleccionado ha sido eliminado.'
148       : `${totalAAborrar} registros seleccionados fueron eliminados.`;
149     this.isToastOpen = true;
150 }
```

```
File Edit Selection View Go Run Terminal Help bio-app
contacto.page.ts M X contacto.page.html M
src > app > pages > contacto > contacto.page.ts > ContactoPage > presentAlert
26 export class ContactoPage implements OnInit {
208
209 /**
210  * Getter que construye el mensaje dinámico de la alerta de confirmación.
211  */
212 get mensajeAlerta(): string {
213     const seleccionados = this.cantidadSeleccionados;
214     return `¿Estás seguro de que deseas eliminar ${seleccionados} registros seleccionados?`;
215 }
216 }
217 }
```



CONTACTO

¡REVISA MI CÓDIGO!

Puedes ver mis proyectos y laboratorios en mi perfil de GitHub.

Nombre

Correo

Comentario del WBC 2026

Et

¿Confirmar acción?

¿Estás seguro de que deseas eliminar 2 registros seleccionados?

SEL



CANCELAR

CONFIRMAR



Buscar por nombre o texto...



Ender Muñoz (endermunoz18@gmail.com)
"Texto Prueba"



Ender (example@gmail.com)
"Texto de Ejemplo"



Leonardo (leoexample@gmail.com)
"Comentario de Prueba"



IR A MI REPOSITORIO

2 registros seleccionados fueron eliminados.



¡REVISA MI CÓDIGO!

Puedes ver mis proyectos y laboratorios en mi perfil de GitHub.

Nombre

Correo

Comentario del WBC 2026

Escribe un mínimo de 5 caracteres...



REGISTRAR DATOS

REGISTROS LOCALES (1)



Seleccionar todos los registros



Buscar por nombre o texto...



Ender (example@gmail.com)
"Texto de Ejemplo"



IR A MI REPOSITORIO



Conclusión

Esta segunda evaluación me permitió llevar la página de contacto de algo estático a una herramienta realmente funcional. Ya no es solo un botón que redirige a GitHub, ahora tiene un formulario que guarda comentarios, una lista que se puede filtrar y la posibilidad de seleccionar y eliminar varios registros a la vez.

Claro, hay que aclarar que todo esto es simulado, porque no tiene conexión con un backend o una base de datos real. Los comentarios se guardan en el localStorage del navegador, así que al recargar la página los datos siguen ahí, pero si se borra la caché o se usa otro dispositivo, no estarán disponibles. Es una forma práctica de simular el registro de datos sin necesidad de montar un servidor.

Entre las cosas que aprendí y puse en práctica están:

- Los formularios reactivos, que me ayudaron a controlar los campos y mostrar mensajes de error cuando algo no estaba bien escrito.
- El almacenamiento local, que me permitió guardar los comentarios en el navegador para que no se pierdan al recargar la página.
- Los getters, que me facilitaron filtrar la lista de comentarios en tiempo real mientras el usuario escribe en la búsqueda.
- Los componentes de Ionic, como los checkboxes, la barra de búsqueda, las alertas y los toasts, que hicieron que la interfaz fuera más interactiva y fácil de usar.
-

El mayor reto fue coordinar el estado de los checkboxes con el almacenamiento local, especialmente el checkbox que selecciona todos los registros, porque tenía que asegurarme de que reflejara correctamente lo que estaba pasando en la lista. Al final, con un poco de lógica y pruebas, logré que todo funcionara de manera limpia y sin errores.

En general, esta evaluación me ayudó a entender mejor cómo construir aplicaciones que no solo se ven bien, sino que también resuelven problemas prácticos, aunque sea de forma simulada, y me dejó una base sólida para seguir desarrollando proyectos más complejos en el futuro.

Link del Repositorio de Github:

https://github.com/EnderLeonardo18/Evaluacion_1_Ionic